图论综合练习

人员

王恩泽、郭栩睿、洪晨栋、洪晨棋 到课

上周作业检查

上周作业链接: https://cppoj.kids123code.com/contest/3728

比赛概况 题目列表 选择题列表 提交记录 实时榜单 选择题排行榜 编辑比赛

王向东老师周六十点半C++floyd

刷新

#	用户名	姓名	编程分	时间	A	B	C	D	E
1	hongchenqi	洪晨棋	400	609	100	100	100	100	
2	wangenze	王恩泽	372	329	100	100	100		72
3	yuxiaolong	于霄龙	350	330	100	100	100	50	
4	yujiarui	于家瑞	300	334	100	100	100		
5	taohuisheng	陶汇笙	300	336	100	100	100		
6	hongchendong	洪晨栋	300	337	100	100	100		
7	guoxurui	郭栩睿	300	347	100	100	100		

本周作业

https://cppoj.kids123code.com/contest/3838 (课上讲了 A ~ C 这些题, 课后作业是 B C 题)

课堂表现

今天课上的 B 题比较复杂一些, 大部分同学课上听懂了但是课上都没有调出来, 课下要争取把这道题调出来。

课堂内容

[USACO08OPEN] Clear And Present Danger S

先用 floyd 求出来任意两点间最短路, 后续每次移动的长度就可以 O(1) 知道了

```
#include <bits/stdc++.h>

using namespace std;

const int maxn = 100 + 5, M = 10000 + 5;
const int inf = 0x3f3f3f3f;
int f[maxn][maxn], w[M];

int main()
{
  int n, m; cin >> n >> m;
  for (int i = 1; i <= m; ++i) cin >> w[i];
```

```
  for (int i = 1; i <= n; ++i) {
    for (int j = 1; j <= n; ++j) cin >> f[i][j];
  }

  for (int k = 1; k <= n; ++k) {
    for (int i = 1; i <= n; ++i) {
      for (int j = 1; j <= n; ++j) f[i][j] = min(f[i][j], f[i][k]+f[k][j]);
    }
  }

  int res = 0;
  for (int i = 2; i <= m; ++i) {
    int last = w[i-1], now = w[i];
    res += f[last][now];
  }
  cout << res << endl;
  return 0;
}
```

Road Blocked

考虑删边操作是比较复杂的, 因为删完边后点与点之间的最短路变化可能变化很大

因此, 可以把整个问题反过来思考, 考虑加边操作

每次加完一条边后, 最短路的更新只可能通过这条边进行跟新, 所以最短路的更新是 O(n^2) 级别的

题目保证, 最多添加 300 条边, 那么这个题就解决了

```
#include <bits/stdc++.h>
#define int long long

using namespace std;

const int N = 300 + 5, M = 2e5 + 5;
const int inf = 0x3f3f3f3f3f3f3f3f;
struct Edge {
  int a, b, c;
} w[N*N];
bool st[N*N];
int f[N][N];

struct node {
  int op, id, x, y;
} q[M];

signed main()
{
  int n, m, Q; cin >> n >> m >> Q;
  for (int i = 1; i <= m; ++i) cin >> w[i].a >> w[i].b >> w[i].c, st[i] = true;
  for (int i = 1; i <= Q; ++i) {
```

```
      cin >> q[i].op;
      if (q[i].op == 1) cin >> q[i].id, st[q[i].id] = false;
      else cin >> q[i].x >> q[i].y;
    }

    memset(f, 0x3f, sizeof(f));
    for (int i = 1; i <= n; ++i) f[i][i] = 0;
    for (int i = 1; i <= m; ++i) {
      int a = w[i].a, b = w[i].b, c = w[i].c;
      if (st[i]) f[a][b] = f[b][a] = c;
    }

    for (int k = 1; k <= n; ++k) {
      for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= n; ++j) f[i][j] = min(f[i][j], f[i][k]+f[k][j]);
      }
    }

    vector<int> ans;
    for (int i = Q; i >= 1; --i) {
      int op = q[i].op, id = q[i].id, x = q[i].x, y = q[i].y;
      if (op == 1) {
        int a = w[id].a, b = w[id].b, c = w[id].c;
        f[a][b] = min(f[a][b], c);
        for (int i = 1; i <= n; ++i) {
          for (int j = 1; j <= n; ++j) {
            int value = min(f[i][a]+f[b][j], f[i][b]+f[a][j]) + f[a][b];
            f[i][j] = min(f[i][j], value);
          }
        }
      } else {
        int dis = (f[x][y]==inf ? -1 : f[x][y]);
        ans.push_back(dis);
      }
    }

    reverse(ans.begin(), ans.end());
    for (int i : ans) cout << i << endl;
    return 0;
  }
```

[CERC1998] 请柬

需要求所有 1 -> i 的最短路和所有 i -> 1 的最短路

1 -> i 的最短路: 可以以 1 为起点用 dijkstra 跑一遍就能求出来

i -> 1 的最短路: 通过建反图, 然后以 1 为起点跑一遍 dijkstra 即可

```
#include <bits/stdc++.h>
#define int long long
```

```cpp
using namespace std;

const int maxn = 2e6 + 5;
const int inf = 0x3f3f3f3f3f3f3f3f;
struct eInfo {
  int to, value;
};
vector<eInfo> vec[maxn];

struct node {
  int id, d;
  bool operator < (const node& p) const { return d < p.d; }
  bool operator > (const node& p) const { return d > p.d; }
};
int dis[maxn];
bool st[maxn];

void dijkstra(int _st) {
  priority_queue<node, vector<node>, greater<node>> q;
  q.push({_st, 0}); dis[_st] = 0;
  while (!q.empty()) {
    node u = q.top(); q.pop();
    int id = u.id, d = u.d;
    if (st[id]) continue;
    st[id] = true;

    for (eInfo it : vec[id]) {
      if (dis[it.to] > d+it.value) {
        dis[it.to] = d+it.value; q.push({it.to, dis[it.to]});
      }
    }
  }
}

signed main()
{
  int n, m; cin >> n >> m;
  while (m -- ) {
    int a, b, c; cin >> a >> b >> c;
    vec[a].push_back({b,c}), vec[n+b].push_back({n+a,c});
  }

  memset(dis, 0x3f, sizeof(dis)), memset(st, false, sizeof(st));
  dijkstra(1), dijkstra(n+1);
  int res = 0;
  for (int i = 2; i <= n; ++i) res += dis[i] + dis[n+i];
  cout << res << endl;
  return 0;
}
```